

Tupo — Local Setup

NGA Communication module · run it on your own laptop · about 30 minutes

Tupo is one repository with five services: `api` (:5190), `realtime` (:5191), `files` (:5192), `worker` (:5193) and the web app (:5194) that the browser talks to. It uses PostgreSQL and Redis. You sign in with your NGA MIS account — the app has no login of its own.

Before you start

Install **Node.js 22 or newer**, **Git**, **PostgreSQL 17** and **Redis**, and get two things from your team lead:

Node.js 22+, Git	nodejs.org · git-scm.com
PostgreSQL 17	enterprisedb.com/downloads/postgres-postgresql-downloads — note the postgres password you set
Redis	Windows: Memurai (memurai.com , free Developer edition) — a Redis-compatible server. macOS: <code>brew install redis && brew services start redis</code>
From your team lead	your NGA MIS login, and the Tupo client secret for <code>SSO_CLIENT_SECRET</code>

1 · Get the code

```
git clone https://github.com/niyongaboemmy/nga-communication-module.git
cd nga-communication-module
npm install
```

One install covers all five services (npm workspaces).

2 · Create your settings files

```
Copy-Item apps\api\.env.example      apps\api\.env
Copy-Item apps\realtime\.env.example apps\realtime\.env
Copy-Item apps\files\.env.example    apps\files\.env
Copy-Item apps\worker\.env.example   apps\worker\.env
Copy-Item apps\web\.env.example       apps\web\.env
Copy-Item packages\db\.env.example    packages\db\.env
```

On macOS or Linux use `cp` with forward slashes for the same six files.

3 · Fill in the settings

Five files share one database URL. In **each** of `apps/api`, `apps/realtime`, `apps/files`, `apps/worker` and `packages/db` set:

<code>DATABASE_URL</code>	<code>postgres://postgres:YOUR_PASSWORD@localhost:5432/tupo_dev</code>
---------------------------	--

Then in `apps/api/.env` only:

<code>NGA_MIS_BASE_URL</code>	<code>https://api.amashuri.com</code> — the template says <code>mis.amashuri.com</code> , which is the web app, not the API; change it
<code>SSO_CLIENT_SECRET</code>	the client secret from your team lead

`JWT_SECRET` must be identical in `api`, `realtime` and `files` — the templates already match, so leave them. `apps/web/.env` needs no changes.

4 · Create the database

```
psql -U postgres -c "CREATE DATABASE tupo_dev"
npm run db:migrate
npm run db:seed
```

psql lives in PostgreSQL's bin folder (Windows: C:\Program Files\PostgreSQL\17\bin), or create the database in pgAdmin. The seed adds the institutional spaces and roles; both commands are safe to re-run.

5 · Run it

```
npm run dev
```

All five services start together (a preflight first checks ports 5190–5194 are free). Open <http://localhost:5194>, click **Sign in**, log in on the MIS page, and you are sent back signed in. <http://localhost:5194/app/system> shows live health of every service. Stop with Ctrl+C.

6 · Working on a feature

Pushing to main deploys to production. Always work on a branch and open a Pull Request:

```
git checkout -b feature/short-description
# make and test your changes, then
git push -u origin feature/short-description
```

If something goes wrong

You see	Do this
<i>password authentication failed for user "postgres"</i>	Fix the password in every DATABASE_URL (5 files).
<i>ECONNREFUSED 127.0.0.1:6379</i>	Redis/Memurai is not running. Start its service.
<i>preflight: port 519x in use</i>	A previous run is still alive. Close that terminal, or <code>netstat -ano findstr :5190</code> , then start again.
<i>Failed to authorize application</i>	Tell your team lead — the MIS needs <code>http://localhost:5194/sso/callback</code> on its allowed list for Tupo. One-time fix on their side.
Sign-in returns HTML / JSON parse error	NGA_MIS_BASE_URL still points at <code>mis.amashuri.com</code> . Set it to <code>https://api.amashuri.com</code> (step 3).
<i>relation ... does not exist</i>	Migrations not applied: <code>npm run db:migrate</code> .
Want a clean database	<code>psql -U postgres -c "DROP DATABASE tupo_dev"</code> , then repeat step 4.
Checks: <code>npm run health · npm test</code> (needs PostgreSQL; uses a throwaway <code>tupo_test</code> database) · <code>npm run typecheck</code>	